Proceedings of IC-NIDC 2025

GG-R1: A FINE-GRAINED RFT APPROACH FOR GUI GROUNDING

Jian Ye[1], *Xin Zhao*[1], *Xuanzhen Feng*[1], *Xiang Gao*[1], *Xin Zhang*[1], *Lanting Li*[1], *Wentao Hong*[2,†]

[1]China United Network Communications Co, Ltd, Beijing, China
[2]Beijing University of Posts and Telecommunications
{yej, zhaoxin60, fengxz11, gaox91, zhangx528, lilt94}@chinaunicom.cn, hwt@bupt.edu.cn

2025 9th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC) | 979-8-3315-9437-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/IC-NIDC67200.2025.11390507

ABSTRACT

Many current GUI agent works primarily focus on supervised fine-tuning based on Vision-Language Models (VLMs). However, this approach not only requires extensive data collection but also demonstrates limited generalization capabilities on out-of-distribution (OOD) tasks. Inspired by DeepSeek-R1, we introduce a rule-based reinforcement fine-tuning methodology for GUI grounding tasks. Specifically, we employ the GRPO algorithm with rule-based rewards (incorporating fine-grained rewards across format, action types, coordinates, length, and difficulty dimensions) and utilize only 152 high-quality samples to fine-tune Qwen2.5-VL-3B, resulting in our model GG-R1-3B. This model achieves 26.8 on Screenspot-Pro, representing a 127% improvement over Qwen2.5-VL-3B (26.8 vs 11.8), and scores 84.3 on ScreenSpot, demonstrating a 29.7% enhancement compared to Qwen2.5-VL-3B (84.3 vs 65.0). Our methodology significantly improves both data utilization efficiency and model performance in GUI action prediction.

Key Words— GUI Agent; Grounding; GPRO; Data-Efficient Fine-Tuning

1. INTRODUCTION

With the advancement of large language models' capabilities, agents based on these models have entered a period of rapid development. These agents can now perform tasks such as deepresearch [1, 2], fixing errors in code repositories [3, 4], and navigating networks [5, 6]. Vision-Language Model (VLM)-based GUI agents can automate task execution on digital devices like humans [7, 8]. Existing GUI agents complete assigned tasks through an observation-planning-execution cycle. However, unlike traditional web agents, the absence of HTML elements forces GUI agents to generate their next actions solely from instructions and screenshots, a process known as GUI grounding. Grounding capability significantly impacts the performance of GUI agents.

Current approaches train grounding models using supervised fine-tuning (SFT) on synthetically generated large-scale datasets [9, 10, 11]. These methods not only incur high data collection costs but also suffer from poor generalization, struggling to maintain performance on out-of-distribution (OOD) domains.Rule-based reinforcement learning and reinforcement fine-tuning (RFT) have emerged as efficient, scalable alternatives to SFT, achieving comparable or superior performance with significantly fewer samples. The success of DeepSeek-R1 in mathematical reasoning demonstrates the paradigm's effectiveness. Recent extensions to multimodal domains have shown notable improvements in object detection [12] and image localization [13], highlighting its cross-modal efficacy in enhancing data efficiency and reducing reliance on large-scale datasets.

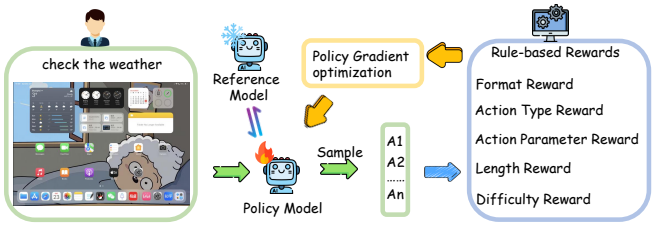

Fig. 1. Overview of the GG-R1 Framework. Given an instruction and a screenshot, the policy model generates n action planning responses with reasoning. A rule-based reward function is then applied, and the policy model is updated using a policy gradient optimization algorithm.

Inspired by UI-R1 [14], we curated 152 small yet high-quality mobile domain data samples based on three criteria: difficulty, diversity, and quality. These data are well-annotated, appropriately challenging, and cover diverse scenarios and instructions. Using these data, our model outperforms traditional SFT methods, demonstrating data efficiency and quality superiority.

We focus on improving action prediction accuracy for low-level instructions (e.g., "open settings").Specifically, we selected two benchmarks: ScreenSpot [15] and ScreenSpot-Pro [16].For each task, we sampled multiple responses from the VLM and evaluated them using our predefined fine-grained reward functions, optimizing the model via GRPO algorithm. Our reward framework includes five compo-

[†] indicates the corresponding author.

979-8-3315-9437-4/25/$31.00 ©2025 IEEE

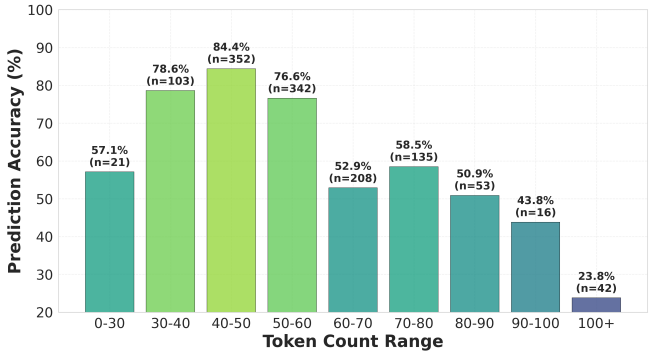

Fig. 2. Relationship between token count and prediction accuracy

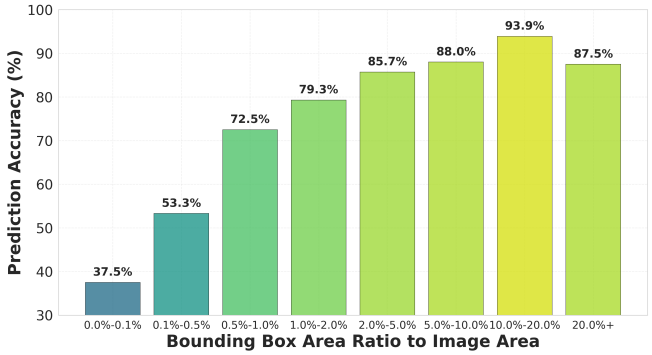

Fig. 3. Relationship between area ratio and prediction accuracy

nents:Action Type Reward,Action Parameter Reward,Format Reward,Length Reward and Difficulty Reward.

In summary,our contributions are:

- We constructed an R1-Zero-style GUI grounding reinforcement learning framework, training GG-R1 to enhance VLM action prediction accuracy in GUI tasks.
- We designed a fine-grained reward function that maximally aligns model improvements with GUI grounding objectives, laying foundations for future research.
- We curated a 152-sample high-quality training dataset that fully exploits RFT potential, reducing data collection costs and improving data efficiency.

2. RELATED WORK

2.1. GUI Agents

LLM-based agents have been explored for autonomous task execution on digital devices [5, 17].Due to the lack of available source code and internal APIs for interaction in many systems and programs, these agents are compelled to evolve into GUI agents.Significant progress has been made in this field through recent research.OS-Atlas [10] achieves performance improvement through GUI grounding pre-training and action fine-tuning. UGround [9] trains a general visual grounding model by collecting a large dataset.UI-TARS [7] UI-TARS is an end-to-end native GUI model.However, these works all rely on the SFT paradigm, which not only requires extensive training data but also limits model generalization capabilities, making it difficult to adapt to unseen graphical interfaces. This limitation has accelerated exploration towards more advanced methodologies.

2.2. Reinforcement Fine-Tuning

Rule-based reinforcement fine-tuning methodology has been rigorously demonstrated effective in Deepseek-R1, where these models exhibited superior performance in mathematical reasoning [18] and code generation [19] domains.By implementing predefined rule-based reward functions, the models achieved enhanced reasoning capabilities during training. Subsequent studies successfully extended this paradigm to multimodal architectures,with VLM-R1 [12] leveraging abundant explicit annotations in vision domains through formalized reward mechanisms to strengthen model reasoning.In the GUI grounding domain, some research improved accuracy in action prediction tasks solely by constructing reward functions based on format compliance and answer correctness. However, we argue that establishing more reasonable and fine-grained reward functions constitutes a critical factor for further improving model performance.

3. METHOD

GG-R1 is an RFT-based method inspired by Deepseek-R1, where we leverage the Generalized Policy Optimization (GRPO) algorithm [18] with rule-based rewards to enhance the performance of vision-language models (VLMs) in GUI action prediction tasks. This work emphasizes three key components: data, algorithm, and reward function definition. Specifically, we collected 152 high-quality, well-annotated samples from the mobile domain, implemented the GRPO algorithm, and designed a fine-grained reward function to train Qwen2.5-VL-3B. Despite utilizing only 0.001% of the data volume (152 vs. 13M), our approach achieves superior performance with 84.3% accuracy on ScreenSpot and 26.8% on ScreenSpot-Pro, surpassing previous state-of-the-art methods such as OS-Atlas.

3.1. Data Selection

A high-quality dataset is crucial for training, and we adhere to the following data collection criteria.

Quality. The instructions within the data must be clear and unambiguous, with annotated results that are accurate.

Difficulty. We first selected the mobile subset from

Table 1. Grounding accuracy on ScreenSpot. The optimal and the suboptimal results are **bolded** and underlined, respectively.

Model	Method	Model Size	Data Size	web		desktop		mobile		Avg
				text	icon	text	icon	text	icon	
Supervised Fine-tuning										
Seeclick	SFT	9.6B	1M	55.7	32.5	72.2	30.0	78.0	52.0	53.4
CogAgent	SFT	18B	222M	70.4	28.6	74.2	20.0	67.0	24.0	47.4
Qwen2.5-VL	SFT	3B	500	78.3	63.1	85.0	46.4	95.2	71.2	75.7
UGround-V1	SFT	7B	10M	80.4	70.4	82.5	63.6	82.8	60.3	73.3
AGUViS	SFT	7B	1M	**88.1**	70.7	85.7	**74.8**	88.3	78.2	81.8
Zero Shot / Reinforcement Learning										
Qwen2-VL	ZS	7B	0	35.2	25.7	76.3	54.3	75.5	60.7	55.3
qwen2.5-VL	ZS	3B	0	60.0	43.2	80.9	40.0	90.5	61.1	65.0
UI-R1	RFT	3B	136	85.2	73.3	90.2	59.3	95.6	84.7	83.3
GG-R1	RFT	3B	152	85.7	**74.3**	**92.3**	60.0	**96.0**	**86.0**	**84.3**

ScreenSpot as the initial dataset, where the low-resolution images ensured moderate difficulty. Subsequently, we employed the Qwen2.5-VL-3B model for sampling, selecting samples where the model's responses were incorrect, which established a baseline difficulty threshold.

Diversity. Beyond text-icon containing data filtered from ScreenSpot, we curated additional samples from ANDROID-CONTROL that exhibit four action types (open_app, scroll, navigation_back, type_text) to ensure diversity.

3.2. Algorithm

Recently, many works have employed the GRPO algorithm with rule-based rewards for reinforcement learning. GRPO calculates advantages by computing the average reward of multiple trajectories sampled within a group, eliminating the need for a value model and reducing bias in reward estimation. The success of this paradigm has been demonstrated by the performance of deepseek-r1.

For a question, we sample N responses $\{o_1, o_2, \ldots, o_N\}$ and use a predefined reward function to calculate their corresponding rewards $\{r_1, r_2, \ldots, r_N\}$. Then GRPO computes their relative advantages. The relative advantage A_i for the i-th response is calculated as:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \ldots, r_N\})}{\text{std}(\{r_1, r_2, \ldots, r_N\})} \tag{1}$$

where mean and std denote the mean and standard deviation operations, respectively.

3.3. Fine-Grained Rewards

Some previous works have solely utilized format validity and answer accuracy as reward functions.However, we argue that a more fine-grained reward function can better enhance model performance and improve generalization capabilities.

As shown in the figure 2, we observed significant accuracy degradation when longer reasoning lengths were involved, and we hypothesize that this stems from model overthinking causing performance deterioration. In figure 3 reveals a strong positive correlation between bounding box ratio and accuracy - as the proportion of bounding box area increases, recognition accuracy improves substantially.

Based on these findings, we specifically designed two additional reward components: length reward and difficulty reward, aiming to mitigate overthinking effects and encourage the model to prioritize learning from challenging samples.

Ultimately, our reward function incorporates five components: action type reward, action parameter reward, format reward, length reward, and difficulty reward. Specifically,

$$R = RT + RP + RF + RL + RD \tag{2}$$

Action Type Reward (RT) evaluates whether the predicted action type matches the ground truth;

Action Parameter Reward (RP) assesses whether the predicted coordinates fall within the ground truth bounding box;

Format Reward (RF) checks if the model generates reasoning processes and answers strictly following the prescribed output format;

Length Reward (RL) assigns higher rewards to response length intervals with proven higher accuracy rates, while imposing negative penalties on excessively long responses. This mechanism suppresses prolonged outputs and mitigates model overthinking;

Difficulty Reward (RD) is defined by the ratio of click bounding box area to image size. We first calculate this ratio, sort the ratios in ascending order, and uniformly generate normalized difficulty scores ranging from 0 to 1. The difficulty reward is only applied when the answer is verified as correct. Additionally, predefined fixed difficulty scores are assigned to specific actions including open_app, scroll, navigation_back, and type_text.

Table 2. Grounding accuracy on ScreenSpot-Pro. The optimal and the suboptimal results are **bolded** and underlined, respectively.* Claude refers to Claude-computer-use.

Model	Method	Dev		Creative		CAD		Scientific		Office		OS		Avg
		text	icon	text	icon	text	icon	text	icon	text	icon	text	icon	
Supervised Fine-tuning														
Seeclick	SFT	0.6	0.0	1.0	0.0	2.5	0.0	3.5	0.0	1.1	0.0	2.8	0.0	1.1
OS-Atlas-4B	SFT	7.1	0.0	3.0	1.4	2.0	0.0	9.0	5.5	5.1	3.8	5.6	0.0	3.7
ShowUI-2B	SFT	16.9	1.4	9.1	0.0	2.5	0.0	13.2	7.3	15.3	7.5	10.3	2.2	7.7
CogAgent-18B	SFT	14.9	0.7	9.6	0.0	7.1	3.1	22.2	1.8	13.0	0.0	5.6	0.0	7.7
Aria-GUI	SFT	16.2	0.0	23.7	2.1	7.6	1.6	27.1	6.4	20.3	1.9	4.7	0.0	11.3
Qwen2.5-VL-3B	SFT	15.6	0.7	13.1	2.1	5.6	3.1	27.8	8.1	20.3	5.7	14.0	0.0	10.8
UGround-7B	SFT	26.6	2.1	27.3	2.8	14.2	1.6	31.9	2.7	31.6	11.3	17.8	0.0	16.5
Claude*	SFT	22.0	3.9	25.9	3.4	14.5	3.7	33.9	15.8	30.1	16.3	11.0	4.5	17.1
OS-Atlas-7B	SFT	**33.1**	1.4	28.8	2.8	12.2	4.7	37.5	7.3	33.9	5.7	27.1	4.5	18.9
Zero Shot / Reinforcement Learning														
Qwen-VL-7B	ZS	0.0	0.0	0.0	0.0	0.0	0.0	0.7	0.0	0.0	0.0	0.0	0.0	0.1
GPT-4o	ZS	1.3	0.0	1.0	0.0	2.0	0.0	2.1	0.0	1.1	0.0	0.0	0.0	0.8
Qwen2-VL-7B	ZS	2.6	0.0	1.5	0.0	0.5	0.0	6.3	0.0	3.4	1.9	0.9	0.0	1.6
Qwen2.5-VL-3B	ZS	14.9	2.1	20.2	1.4	4.1	4.7	34.0	7.3	22.0	3.8	6.5	2.2	11.8
UI-R1-3B	RL	22.7	**4.1**	27.3	**3.5**	11.2	**6.3**	42.4	11.8	32.2	11.3	13.1	4.5	17.8
GG-R1-3B	RL	32.5	3.4	**35.4**	2.1	**27.4**	3.1	**56.2**	**17.3**	**52.5**	**17.0**	**28.0**	**7.9**	**26.8**

4. EXPERIMENTS

We selected Qwen2.5-VL-3B-Instruct as the base model. Using the aforementioned 152 carefully constructed data samples and RFT methodology, we conducted 8 training epochs through the Open-R1 framework [20], ultimately developing GG-R1-3B. All experiments were performed on 8×NVIDIA H800-80G GPUs. Training required approximately 5 hours. To ensure fairness, inference results for other models were sourced from the UI-R1 paper.

We evaluated our model using ScreenSpot [15] and ScreenSpot-Pro [16]. ScreenSpot assesses GUI grounding capabilities across mobile, desktop, and web platforms, while ScreenSpot-Pro focuses on high-resolution professional environments. During evaluation, a predicted click coordinate was deemed correct if it fell within the ground-truth bounding box. Accuracy was calculated as the ratio of correctly predicted samples to the total number of test samples.

As shown in the table 1, GG-R1-3B achieved a score of 84.3 on ScreenSpot, which not only significantly outperforms Qwen2.5-VL-3B, but also surpasses 7B parameter SFT models (such as UGround-V1 and AGUVIS) trained on substantially larger datasets.

As shown in the table 2, our results further demonstrate that GG-R1-3B obtains 26.8 on ScreenSpot-Pro, exceeding UI-R1 (which employed the same RFT methodology). Notably, GG-R1-3B outperforms the previous state-of-the-art method OS-Atlas (26.8 vs 18.9) with merely 0.001% of the data volume (152 vs. 13M).This further confirms that our difficulty reward mechanism more effectively enhances the model's capability to learn from challenging samples.

Significantly, our approach achieves these results with only 152 data samples. This massive disparity in data volume highlights the effectiveness and data efficiency of our methodology, demonstrating the potential of reinforcement learning approaches as data-efficient and scalable solutions for model training in resource-constrained environments.

5. CONCLUSION

We propose GG-R1, a method that employs rule-based reinforcement learning to enhance model performance in GUI action prediction tasks. We designed a fine-grained reward function that not only evaluates answer accuracy but also takes into account factors such as question difficulty and reasoning length. With only 152 training samples, our approach achieves significant performance improvements, particularly in high-resolution scenarios. Our method requires minimal data while demonstrating strong generalization capabilities, offering a scalable alternative to traditional SFT.

6. ACKNOWLEDGEMENTS

This work is supported in part by China United Network Communications Co Ltd, Beijing, 102600, China,and in part by Beijing University of Posts and Telecommunications.

7. REFERENCES

[1] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han, "Search-r1: Training llms to reason and leverage search engines with reinforcement learning," 2025.

[2] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu, "Deepresearcher: Scaling deep research via reinforcement learning in real-world environments," 2025.

[3] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen, "Swe-fixer: Training open-source llms for effective and efficient github issue resolution," 2025.

[4] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang, "Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution," 2025.

[5] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su, "Mind2web: Towards a generalist agent for the web," 2023.

[6] Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong, "Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning," 2025.

[7] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi, "Ui-tars: Pioneering automated gui interaction with native agents," 2025.

[8] Yanheng He, Jiahe Jin, and Pengfei Liu, "Efficient agent training for computer use," 2025.

[9] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su, "Navigating the digital world as humans do: Universal visual grounding for gui agents," 2025.

[10] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao, "Os-atlas: A foundation action model for generalist gui agents," 2024.

[11] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong, "Aguvis: Unified pure vision agents for autonomous gui interaction," 2025.

[12] Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, Ruochen Xu, and Tiancheng Zhao, "Vlm-r1: A stable and generalizable r1-style large vision-language model," 2025.

[13] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin, "Vision-r1: Incentivizing reasoning capability in multimodal large language models," 2025.

[14] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanjing Xiong, and Hongsheng Li, "Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning," 2025.

[15] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu, "Seeclick: Harnessing gui grounding for advanced visual gui agents," 2024.

[16] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua, "Screenspot-pro: Gui grounding for professional high-resolution computer use," 2025.

[17] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig, "Webarena: A realistic web environment for building autonomous agents," 2024.

[18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," 2024.

[19] Jiawei Liu and Lingming Zhang, "Code-r1: Reproducing r1 for code with reliable rewards," *arXiv preprint arXiv:2503.18470*, vol. 3, 2025.

[20] Hugging Face, "Open r1: A fully open reproduction of deepseek-r1," January 2025.